Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики і програмної інженерії

Звіт

з дисципліни «Основи розробки програмного забезпечення на платформі Java»
Комп'ютерний практикум №5.
Механізм інтерфейсів мови Java та його використання для обробки подій користувача

Виконав: гр. ІП-з41 Черепов Олександр Павлович

Перевірив: старший викладач Дифучина Олександра Юріївна

Оцінка: _____

Дата оцінки: _____

2026

Комп’ютерний практикум №5

Тема роботи: Механізм інтерфейсів мови Java та його використання для обробки подій користувача.
Мета роботи: Дослідити механізм інтерфейсів класу.

Завдання до роботи

1. Розробити інтерфейс користувача до моделі предметної області, що розроблялась на попередніх заняттях, з використанням компонентів бібліотеки swing. 40 балів.

Застосувати інтерфейс для:

- задання та корегування параметрів об’єктів, які створюються при розробці моделі (з перевіркою введеного значення на приналежність до допустимих)
- задання та корегування параметрів моделі
- виведення повідомлень при неправильній побудові моделі
- управління обчисленнями
- виведення результатів моделювання

2. Додати події, пов’язані з періодичністю, та розробити для них програмний код з використанням класу Timer. 20 балів.

3. Виконати тестування програми. 20 балів.

4. Дослідити застосування лямбда-виразів та посилань на методи при програмуванні обробників подій. Сформулювати переваги та недоліки цих способів у порівнянні з анонімними класами. 20 балів.

Номер варіанту: 6.

Реалізація

Структура файлів:

```
programme/
├── auditory/
│   └── Auditory.java
├── employee/
│   ├── Employee.java
│   └── Laborant.java
├── managers/
```

```
│   ├── AuditoriesMap.java
│   └── EmployeesCollection.java
├── Main.java
└── MainFrame.java
```

Лістинг програми (лише нові/змінені файли):

programme /auditory/Auditory.java (++):

```
@Override
public String toString() {
return "ID: " + this.getId() + " - " + this.getForSubject() +
      " (c:" + this.getCapacity() + " f: " + this.getFloor() + ")";
}
```

programme/managers/AuditoryMap.java (--):

```
package managers;
import auditory.Auditory;

import java.util.HashMap;
import java.util.Map;

public class AuditoriesMap {
    private Map<Integer, Auditory> auditoriesMap;

    public AuditoriesMap() {
        auditoriesMap = new HashMap<>();
    }

    public void addAuditory(Auditory a) {
        auditoriesMap.put(a.getId(), a);
    }

    public Map<Integer, Auditory> getAuditoriesMap() {
        return auditoriesMap;
    }
}
```

programme/managers/EmployeesCollection.java (+-):

```
package managers;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;

import employee.Employee;
```

```
public class EmployeesCollection {
    private Collection<Employee> employeesList;

    public EmployeesCollection() {
        employeesList = new ArrayList<>();
    }

    public EmployeesCollection(boolean uniqueOnly) {
        if (uniqueOnly) {
            employeesList = new HashSet<>();
        } else {
            employeesList = new ArrayList<>();
        }
    }

    public void addEmployee(Employee e) {
        boolean wasAdded = employeesList.add(e);

        if (!wasAdded) {
            throw new IllegalArgumentException("This employee is already in
the collection.");
        }
    }

    public Collection<Employee> getEmployeesList() {
        return employeesList;
    }

    public Employee findEmployeeById(int id) {
        for (Employee employee : employeesList) {
            if (employee.getId() == id) {
                return employee;
            }
        }

        return null;
    }

    public List<Employee> sortEmployeesById() {
        List<Employee> sortedList = new ArrayList<>(employeesList);
        sortedList.sort(Comparator.comparingInt(Employee::getId));

        return sortedList;
    }
}
```

programme/Main.java (+-):

```
import javax.swing.SwingUtilities;

import auditory.Auditory;
```

```
import employee.Employee;
import employee.Laborant;
import managers.AuditoriesMap;
import managers.EmployeesCollection;

public class Main {
    public static void main(String[] args) {
        Auditory a1 = new Auditory(1, "Informatics", 70);
        Auditory a2 = new Auditory(2, "Mathematics", 30);
        Auditory a3 = new Auditory(1, "Physics", 50);

        Employee emp0 = new Employee();
        Employee emp1 = new Employee("Jane Smith", 6000, new Auditory[]{a2});
        Employee lab1 = new Laborant("Alice Johnson", 7000, new
Auditory[]{a2, a3}, 2);
        Employee lab2 = new Laborant("Bob Brown", 5500, new Auditory[]{a1},
1);
        Employee lab3 = new Laborant("Charlie Davis", 6500, new
Auditory[]{a1, a2}, 2);

        EmployeesCollection collection = new EmployeesCollection(true);
        collection.addEmployee(emp0);
        collection.addEmployee(emp1);
        collection.addEmployee(lab1);
        collection.addEmployee(lab2);
        collection.addEmployee(lab3);

        AuditoriesMap auditoriesMap = new AuditoriesMap();
        auditoriesMap.addAuditory(a1);
        auditoriesMap.addAuditory(a2);
        auditoriesMap.addAuditory(a3);

        SwingUtilities.invokeLater(() -> {
            MainFrame frame = new MainFrame(collection, auditoriesMap);
            frame.setVisible(true);
        });
    }
}
```

programme/MainFrame.java:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

import javax.swing.table.DefaultTableModel;

import auditory.Auditory;

import java.util.Collection;
```

```
import employee.Employee;
import employee.Laborant;
import managers.AuditoriesMap;
import managers.EmployeesCollection;

public class MainFrame extends JFrame {
    private EmployeesCollection collection;
    private AuditoriesMap auditoriesMap;
    private JTable table;
    private DefaultTableModel tableModel;
    private JList<Auditory> auditoryList;
    private Timer simulationTimer;

    public MainFrame(EmployeesCollection collection, AuditoriesMap
auditories) {
        this.collection = collection;
        this.auditoriesMap = auditories;

        setTitle("Staff Management System");
        setSize(800, 400);
        setResizable(false);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        initTable();
        initAuditoriesList();
        initPanel();

        updateTable();
    }

    public void initTable() {
        String[] columns = {"ID", "Name", "Salary", "Role", "Auditories",
"Master Key"};

        tableModel = new DefaultTableModel(columns, 0) {
            @Override
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };

        table = new JTable(tableModel);
        JScrollPane scrollPane = new JScrollPane(table);
        add(scrollPane, BorderLayout.CENTER);
    }

    public void updateTable() {
        tableModel.setRowCount(0);
```

```
        Collection<Employee> currentList = collection.sortEmployeesById();

        for (Employee e : currentList) {
            String role = e.getClass().getSimpleName();
            String auditoriesText = "-";
            String masterKeyText = "-";

            if (e.getAuditories() != null) {
                auditoriesText = "";
                for (Auditory a : e.getAuditories()) {
                    String auditoryId = String.valueOf(a.getId()) + " ";
                    auditoriesText = auditoriesText + auditoryId;
                }
            }

            if (e instanceof Laborant) {
                Laborant lab = (Laborant) e;
                int key = lab.getMasterKeyId();
                if (key >= 1) {
                    masterKeyText = String.valueOf(key);
                }
            }

            Object[] row = {
                e.getId(),
                e.getName(),
                String.format("$%.2f", e.getSalary()),
                role,
                auditoriesText,
                masterKeyText
            };

            tableModel.addRow(row);
        }
    }

    public JList<Auditory> initAuditoriesList() {
        Collection<Auditory> values =
auditoriesMap.getAuditoriesMap().values();
        Auditory[] availableAuditories = values.toArray(new Auditory[0]);

        auditoryList = new JList<>(availableAuditories);
        auditoryList.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SE
LECTION);

        return auditoryList;
    }

    public void initPanel() {
        JPanel topPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
```

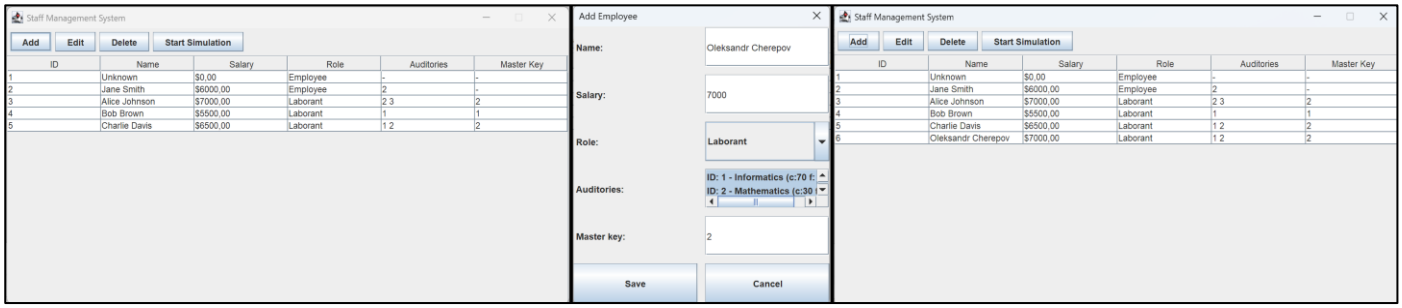
```
        JButton addButton = new JButton("Add");
        JButton editButton = new JButton("Edit");
        JButton deleteButton = new JButton("Delete");
        JButton simulationButton = new JButton("Start Simulation");

        addButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                openEmployeeDialog(null);
            }
        });

        editButton.addActionListener(e -> {
            Employee empToEdit = getSelectedEmployee();
            if (empToEdit != null) {
                openEmployeeDialog(empToEdit);
            }
        });

        deleteButton.addActionListener(this::handleDeleteAction);

        simulationTimer = new Timer(2000, e -> {
            for (Employee emp : collection.getEmployeesList()) {
                emp.giveRaise(5, true);
            }
            updateTable();
        });

        simulationButton.addActionListener(e -> {
            if (simulationTimer.isRunning()) {
                simulationTimer.stop();
                simulationButton.setText("Start Simulation");
            } else {
                simulationTimer.start();
                simulationButton.setText("Stop Simulation");
            }
        });

        topPanel.add(addButton);
        topPanel.add(editButton);
        topPanel.add(deleteButton);
        topPanel.add(simulationButton);

        add(topPanel, BorderLayout.NORTH);
    }

    private void handleDeleteAction(ActionEvent e) {
        Employee empToDelete = getSelectedEmployee();
        if (empToDelete != null) {
            collection.getEmployeesList().remove(empToDelete);
```

```
            updateTable();
        }
    }

    private Employee getSelectedEmployee() {
        int selectedRow = table.getSelectedRow();

        if (selectedRow == -1) {
            JOptionPane.showMessageDialog(this, "Please select an employee.",
"No Selection", JOptionPane.WARNING_MESSAGE);
            return null;
        }

        String id = (String) tableModel.getValueAt(selectedRow, 0);
        return collection.findEmployeeById(id);
    }

    private void openEmployeeDialog(Employee employeeToEdit) {
        boolean isEditMode = (employeeToEdit != null);
        String title = isEditMode ? "Edit Employee" : "Add Employee";

        JDialog dialog = new JDialog(this, title, true);
        dialog.setSize(350, 400);
        dialog.setResizable(false);
        dialog.setLocationRelativeTo(this);
        dialog.setLayout(new GridLayout(6, 2, 10, 10));

        JLabel nameLabel = new JLabel(" Name:");
        JTextField nameField = new JTextField(isEditMode ?
employeeToEdit.getName() : "");

        JLabel salaryLabel = new JLabel(" Salary:");
        JTextField salaryField = new JTextField(isEditMode ?
String.valueOf(employeeToEdit.getSalary()) : "");

        JLabel roleLabel = new JLabel(" Role:");
        JComboBox<String> roleBox = new JComboBox<>(new String[]{"Employee",
"Laborant"});

        JLabel auditoriesLabel = new JLabel(" Auditories:");
        JList<Auditory> auditoryList = initAuditoriesList();
        JScrollPane listScroller = new JScrollPane(auditoryList);

        JLabel keyLabel = new JLabel(" Master key:");
        JTextField keyField = new JTextField();

        if (isEditMode) {
            boolean isLaborant = employeeToEdit instanceof Laborant;
            roleBox.setSelectedItem(isLaborant ? "Laborant" : "Employee");
            roleBox.setEnabled(false);
```

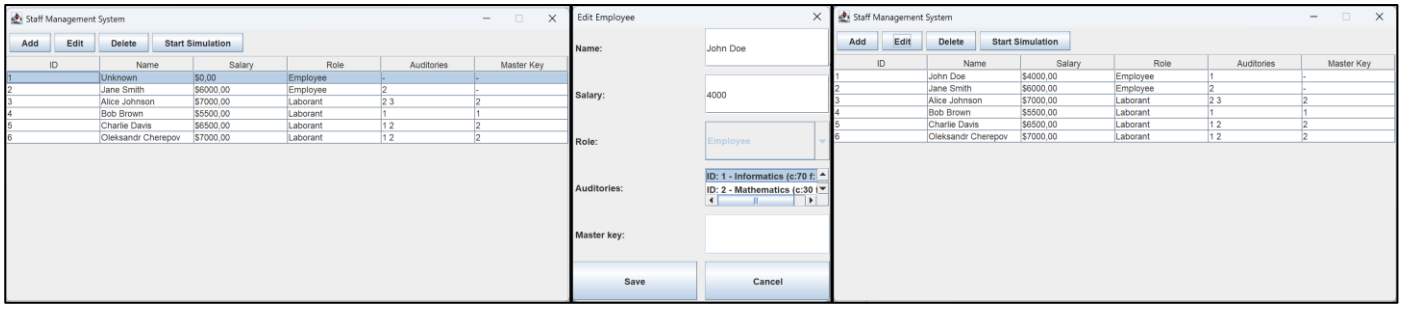
```
            if (isLaborant) {
                keyField.setText(String.valueOf(((Laborant)
employeeToEdit).getMasterKeyId()));
            }
            keyField.setEnabled(isLaborant);
        } else {
            keyField.setEnabled(false);
            roleBox.addActionListener(e -> {
                boolean laborantSelected =
roleBox.getSelectedItem().equals("Laborant");
                keyField.setEnabled(laborantSelected);
                if (!laborantSelected) keyField.setText("");
            });
        }

        JButton saveButton = new JButton("Save");
        JButton cancelButton = new JButton("Cancel");

        saveButton.addActionListener(e -> {
            try {
                String name = nameField.getText();
                double salary = Double.parseDouble(salaryField.getText());
                Auditory[] selectedAuditories =
auditoryList.getSelectedValuesList().toArray(new Auditory[0]);
                Integer masterKey = keyField.getText().isEmpty() ? null :
Integer.parseInt(keyField.getText());

                Employee targetEmployee = isEditMode ? employeeToEdit : null;

                if (!isEditMode) {
                    if (roleBox.getSelectedItem().equals("Laborant")) {
                        targetEmployee = new Laborant();
                    } else {
                        targetEmployee = new Employee();
                    }
                }

                targetEmployee.setName(name);
                targetEmployee.setSalary(salary);
                targetEmployee.setAuditories(selectedAuditories);

                if (targetEmployee instanceof Laborant) {
                    ((Laborant) targetEmployee).setMasterKeyId(masterKey !=
null ? masterKey : -1);
                }

                if (!isEditMode) {
                    collection.addEmployee(targetEmployee);
                }

                updateTable();
```

```
                dialog.dispose();

            } catch (IllegalArgumentException ex) {
                JOptionPane.showMessageDialog(dialog, ex.getMessage(), "Input
Error", JOptionPane.ERROR_MESSAGE);
            }
        });

        cancelButton.addActionListener(e -> dialog.dispose());

        dialog.add(nameLabel);       dialog.add(nameField);
        dialog.add(salaryLabel);     dialog.add(salaryField);
        dialog.add(roleLabel);       dialog.add(roleBox);
        dialog.add(auditoriesLabel); dialog.add(listScroller);
        dialog.add(keyLabel);        dialog.add(keyField);
        dialog.add(saveButton);      dialog.add(cancelButton);

        dialog.setVisible(true);
    }
}
```

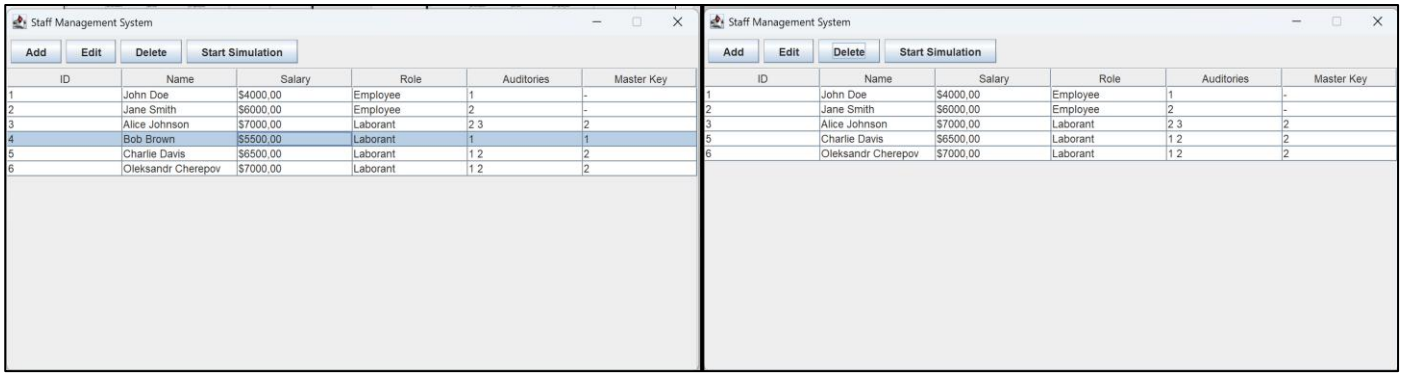
Результати тестування програми (додавання об’єкту – до/в процесі/після):



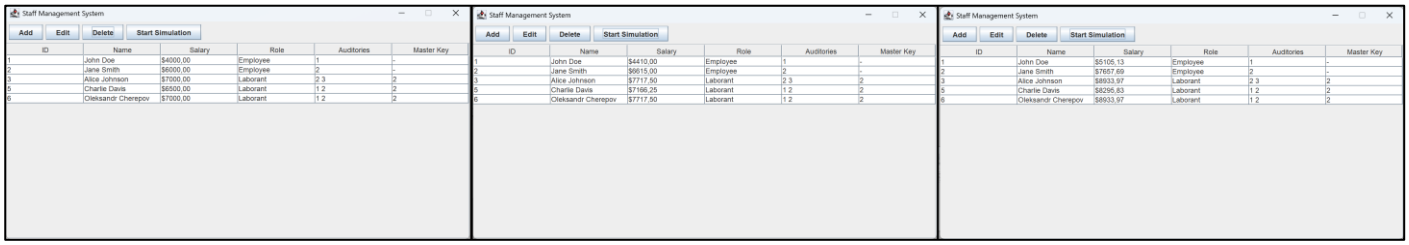
Результати тестування програми (корегування параметрів – до/в процесі/після):



Результати тестування програми (видалення об’єкту – до/після):



Результати тестування програми (запуск симуляції – 0/4/10 секунд):

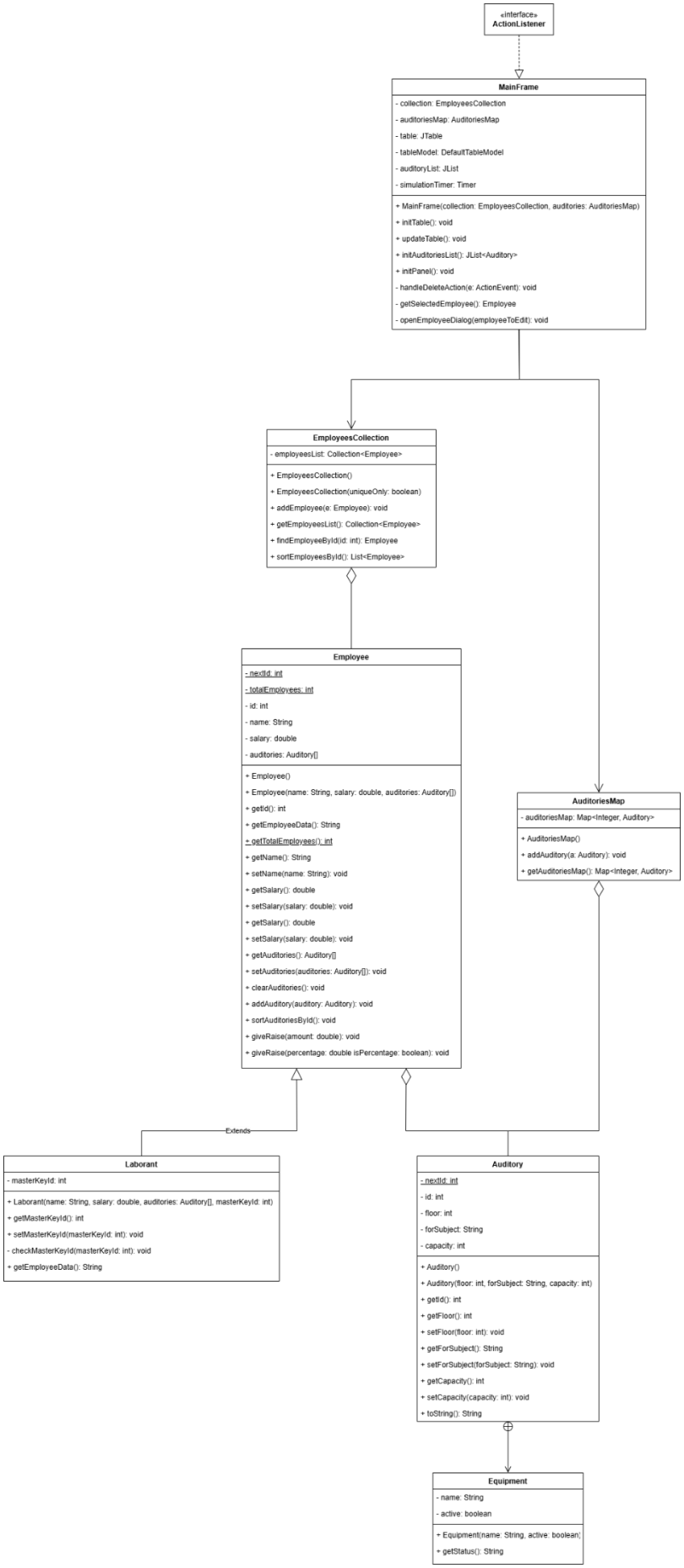


Результат досліження механізму інтерфейсів:

Під час виконання роботи було досліджено механізм інтерфейсів (зокрема ActionListener), який є основою обробки подій у Java. Головним результатом дослідження стало розуміння наступних аспектів використання інтерфейсів:

1. Прихована імплементація: Було з'ясовано, що для обробки подій не обов'язково використовувати ключове слово *implements* у сигнатурі головного класу.
2. Анонімні класи: У класичному підході імплементація відбувається через пряме створення анонімного класу *new ActionListener()*.
3. Функціональні інтерфейси (Лямбди та посилання на методи): Найважливішим відкриттям стало те, що сучасна Java дозволяє "сховати" факт імплементації інтерфейсу від програміста. Компілятор автоматично перетворює конструкції типу *e -> {...}* або *this::handleAction* на повноцінні об'єкти, що імплементують цільовий інтерфейс. Це робить код значно чистішим і позбавляє від зайвого візуального шуму.

Діаграма класів:



Висновок

Під час виконання комп'ютерного практикуму було розроблено графічний інтерфейс користувача з використанням бібліотеки Swing та реалізовано обробку подій на основі механізму інтерфейсів.

На практиці було досліджено еволюцію підходів до реалізації функціональних інтерфейсів та доведено, що хоча Java пропонує кілька шляхів (анонімні класи, лямбда-вирази та посилання на методи), найзручнішим та найоптимальнішим способом обробки подій є використання лямбда-виразів. Цей підхід забезпечує високу читабельність коду та дозволяє уникнути плутанини з передачею параметрів, яка часто виникає при використанні посилань на методи (де вимагається неочевидний строгий збіг сигнатур). Крім того, колбеки відрізняються максимальною універсальністю: розробник отримує гнучкість писати коротку логіку безпосередньо всередині самого виразу, або ж за необхідності легко винести її в окрему функцію, повністю контролюючи те, які саме аргументи та куди передаються.